

Assignment 2

Reading and Writing a Records Document

Problem 2.2.6

Student: Anish Shirodkar

Runtime: Node.js v20.20.2

Testing framework: Jasmine

1 Program Overview

The program reads a records document from a local text file, checks its structure and property values, reports errors with line numbers, and sorts records in ascending order of their **TIME** values.

When validation succeeds, the sorted records are written to a new file. When validation fails, no output file is written. Existing output files are not overwritten.

Command-line usage

```
node records.js <input-file> [output-file] [registry-file]
```

The default output filename is **sorted.txt**. The optional registry file contains previously used identifiers, one per line.

2 Automated Test Results

The following output was captured from the Jasmine test run:

```
> assignment-201.0.0 test
> jasmine

Randomized with seed 29596
Started
.....

47 specs, 0 failures
Finished in 0.202 seconds
Randomized with seed 29596
(jasmine --random=true --seed=29596)
```

The tests cover date and time validation, weight and unit validation, colors, record parsing, identifier conflicts, formatting errors, sorting, and command-line file operations.

3 Sorted Output

The input contains ten records with different dates, optional properties, and mixed-case keywords. The following command produced the output file:

```
node records.js input.txt sorted.txt

Successfully validated and sorted 10 records.
Saved to: sorted.txt
```

The contents of `sorted.txt` are shown below. Property names use a consistent order, while property values retain their original capitalization.

Records 1–5

```
BEGIN:RECORD
IDENTIFIER: A-005
TIME: 20240229T120000
COLOR: green
END:RECORD

BEGIN:RECORD
IDENTIFIER: A-007
TIME: 20241231T235959
WEIGHT: 0.5
UNITS: kilograms
COLOR: #FF0000
END:RECORD

BEGIN:RECORD
IDENTIFIER: A-009
TIME: 20250101T000000
COLOR: black
END:RECORD

BEGIN:RECORD
IDENTIFIER: A-006
TIME: 20250927T080000
WEIGHT: 0
UNITS: KILOGRAMS
END:RECORD

BEGIN:RECORD
IDENTIFIER: a-003
TIME: 20250927T093005
COLOR: rgb(255, 128, 0)
END:RECORD
```

Records 6–10

```
BEGIN:RECORD  
IDENTIFIER: A-002  
TIME: 20250927T100000  
WEIGHT: 160  
UNITS: pounds  
COLOR: blue  
END:RECORD
```

```
BEGIN:RECORD  
IDENTIFIER: A-008  
TIME: 20250927T100000  
WEIGHT: 125.75  
UNITS: Pounds  
COLOR: rgb(0, 0, 255)  
END:RECORD
```

```
BEGIN:RECORD  
IDENTIFIER: A-001  
TIME: 20250927T101530  
WEIGHT: 72.4  
UNITS: kilograms  
COLOR: #2F54EB  
END:RECORD
```

```
BEGIN:RECORD  
IDENTIFIER: A-010  
TIME: 20251001T183045  
WEIGHT: 85  
UNITS: kilograms  
COLOR: white  
END:RECORD
```

```
BEGIN:RECORD  
IDENTIFIER: A-004  
TIME: 20260101T000000  
END:RECORD
```

Observations

- The records are ordered from February 29, 2024, through January 1, 2026.
- Records A-002 and A-008 have identical timestamps and retain their input order.
- Records containing only the required properties are accepted.
- Zero weight and the valid leap-day date are accepted.

4 Error Reporting

These demonstrations use deliberately invalid inputs. The program reports the detected errors and does not write an output file.

4.1 Structure and Required Properties

```
node records.js invalid.txt

Found 4 error(s):
Line 5: Duplicate property COLOR.
Line 6: Unknown property "SIZE".
Line 1: Missing required property IDENTIFIER.
Line 1: Missing UNITS for a record with WEIGHT.
No output file was written.
```

All four errors are reported during the same run. For a missing property, the reported line identifies the start of the affected record.

4.2 Invalid Dates and Times

```
node records.js invalid-time.txt

Found 2 error(s):
Line 3: Invalid TIME "20250230T101530"; expected a real date and time in YYYYMMDDTHHMMSS
format.
Line 8: Invalid TIME "20250927T250000"; expected a real date and time in YYYYMMDDTHHMMSS
format.
No output file was written.
```

The first value contains February 30, which is not a valid calendar date. The second value uses hour 25, outside the accepted range of 00–23.

4.3 Invalid Weights and Units

```
node records.js invalid-weight.txt

Found 3 error(s):
Line 4: Invalid WEIGHT "-5"; expected a finite, nonnegative decimal number.
Line 5: Invalid UNITS "grams"; expected "kilograms" or "pounds".
Line 11: Invalid WEIGHT "heavy"; expected a finite, nonnegative decimal number.
No output file was written.
```

5 Cross-File Identifier Check

The optional identifier registry used for this demonstration contains:

```
A-001
previous-record-100
```

The program checks the input identifiers against this registry:

```
node records.js input.txt registry-check.txt known-identifiers.txt

Found 1 error(s):
Line 2: IDENTIFIER "A-001" already exists in the supplied identifier registry.
No output file was written.
```

Scope and Limitation

The program detects duplicate identifiers within the current input and conflicts with identifiers listed in the supplied registry. Identifier values are compared case-sensitively.

The registry is optional and is not automatically updated. Therefore, this implementation does not independently guarantee global uniqueness against records absent from the registry. Cross-file checking depends on the completeness of the supplied registry.

6 Implementation Conventions

- **Date and time:** Values use YYYYMMDDTHHMMSS, with years 0001–9999, valid calendar dates, hours 00–23, and minutes and seconds 00–59.
- **Weight:** Finite, nonnegative decimal values are accepted, including zero. Units must be kilograms or pounds, compared without regard to capitalization.
- **Color:** Accepted values are names in the predefined color list, three- or six-digit hexadecimal codes, and RGB triples with integer components from 0 to 255.
- **Multiple properties:** Within a property value, whitespace followed by a name and colon is interpreted as an additional property. This is a parsing convention; unrestricted text values containing that pattern are ambiguous.
- **Output:** Records are written only after validation succeeds. An existing destination file is never overwritten.